

Object Oriented Programming with Java

Table of Contents

Chapter 1: Variables and Data Types.....	3
1: Variables	3
2: Data Types	3
Chapter 2: Function	5
1: Hàm là gì?	5
2: Các loại hàm.....	5
3: Overload/Overloading.....	6
Chapter 3: Object and Class.....	6
1: Object	6
2: Class.....	7
3: Constructor – Phễu.....	8
4: Access Specifier/Access Modifier	9
5: Object lưu trữ trong ram như thế nào?	9
Chapter 4: Static.....	10
Chapter 5: Input & Try-Catch	11
1: Input	11

2: Try-Catch	12
Chapter 6: Array	13
1: Primitive Array	13
2: Object Array	14
Chapter 7: Inheritance	15
1: Giới thiệu tổng quát.....	15
2: Override	16
3: Cách tổ chức vùng ram khi chơi với Inheritance	17
4: Abstract Class	19
5: Anonymous Class	21
Chapter 8: Interface	22
Chapter 9: OOP CONCEPTS / OOP PRINCIPLES / FUNDAMENTAL CONCEPTS OF OOP / FOUR PILLARS OF OOP	27
1: Abstraction – Trừu tượng hóa mọi vật.....	27
2: Encapsulation – Cô lập đóng gói mọi thứ	28
3: Inheritance – Tái sử dụng / có xài luôn	29
4: Polymorphism – Đa hình / đa ánh xạ - một ra nhiều	30
Chapter 10: Collections Framework.....	31
1: Giới thiệu tổng quát.....	31
2: Các loại tử.....	31
Chapter 11: Wrapper Class	33

1: Giới thiệu tổng quát.....	33
2: Boxing & Unboxing	35
3: Immutable Class & Mutable Class.....	35
4: So sánh 2 object.....	36
5: Pool	37

Chapter 1: Variables and Data Types

1: Variables

- Biến là một vùng ram được đặt tên/định danh/identifier chiếm một số byte nhất định tùy hình thái dữ liệu mà nó sẽ chứa, dùng để chứa một value nào đó.
- Biến là cách đặt tên cho một đại lượng, một giá trị.

int a = 10;

2: Data Types

- Hình thái dữ liệu được gọi là data type, gồm hai loại: primitive, object.
- Primitive data types:

Kiểu dữ liệu	Số bit	Giá trị nhỏ nhất	Giá trị lớn nhất
byte	8	-128 (-2^7)	127 (2^7-1)
short	16	-32,768 (-2^{15})	32,767 ($2^{15}-1$)
int	32	-2,147,483,648 (-2^{31})	2,147,483,647 ($2^{31}-1$)
long	64	-9,223,372,036,854,775,808 (-2^{63})	9,223,372,036,854,775,807 ($2^{63}-1$)
float	32	$-3.4028235 \times 10^{38}$	3.4028235×10^{38}
double	64	$-1.7976931348623157 \times 10^{308}$	$1.7976931348623157 \times 10^{308}$
boolean	false true		
char	16	'\u0000' (0)	'\uffff' (65,535).

- Mọi giá trị/value xuất hiện trong code, số nào đó, chữ nào đó, chuỗi nào đó, 3.14, “age”, ... xuất hiện trong code được gọi là literal value.
- Mọi literal value của số nguyên hay mọi số nguyên xuất hiện trong code đều được xem là kiểu int – 4 bytes.

int a = 79;

long money = 2_200_000_000L;

int status = 0xFA; // Hệ hexa: lấy FA (hex) => 250 (dec) in ra.

int phone = 070; // Hệ oct: lấy 70 (oct) => 56 (dec) in ra.

- Java ưu tiên chơi với double hơn là chơi với float. Mọi số thực xuất hiện trong code đều được xem là kiểu double – 8 bytes.

double pi = 3.14;

float vat = 0.1F;

- Trong Java, kiểu char xài 2 bytes.

char unit = '\$';

- Java cung cấp 1 data type hoàn toàn mới, dùng để lưu trữ chuỗi: String. String dùng để lưu trữ, tên gọi cho những thứ phức tạp, một đồng các kí tự ghép lại với nhau, kiểu object.

String name = “Ngô Huỳnh Tấn Lộc”;

- Một biểu thức so sánh, một mệnh đề, một câu phát biểu sẽ rơi vào 2 trạng thái: đúng – sai. Java quy ước đúng là true, sai là false => kiểu boolean (1 bit, 1 byte).

boolean marriedStatus = false;

Chapter 2: Function

1: Hàm là gì?

Hàm/Function – Method/Phương Thức		
Hàm là gì? Một quy tắc/cách thức xử lí đầu vào để có đầu ra		$y = f(x) = x^2$ SinhTổ = máyXayÉp (tráiCâyĐưaVào) = nghiền/ép/trộn
Sử dụng hàm Gọi tên em/hàm với data vào/ra		$y = f(2) = 2^2 = 4$ $y = f(3) = 3^2 = 9$ SinhTổ-Cam = máyXayÉp (cam) SinhTổ-Cà = máyXayÉp (càRốt, càPháo, càPhê)
Bên trong hàm có gì?/Cấu tạo hàm		$y = f(x) = x^2$

© 2021 – Nguyễn Thế Hoàng – FPT University

2: Các loại hàm

02 Hàm/Function – Method/Phương Thức (tt.)				
SinhTổ	=	máyXayÉp (tráiCâyĐưaVào)	=	nghiên/ép/trộn
y	=	f (x)	=	x ²
dầu ra		tên hàm		thân hàm/xử lí
output		function name		body of function
returned value		method name		implement of function
không-ra f(không-vào)		không-ra f(có-vào)		
<pre>void f(void) { cần scanf() để có data mà xử lí; cần printf() để in kết quả đã xử lí; }</pre>		<pre>void f(int a) { không nên scanf() vì đã có data a đưa vào để xử lí; cần printf() để in kết quả đã xử lí; }</pre>		
có-ra f(không-vào)		có-ra f(có-vào) //double r = sqrt(4); Math.sqrt(4);		
<pre>int f(void) { cần scanf() để có data mà xử lí; không nên printf() vì đã return data ra ngoài; bắt buộc return xxx-value ra ngoài qua tên hàm; } //Tên hàm là 1 biến được gán value từ return</pre>		<pre>int f(int a) { //IPO không nên scanf() vì đã có data a đưa vào để xử lí; không nên printf() vì đã return data ra ngoài; bắt buộc return xxx-value ra ngoài qua tên hàm; } //Tên hàm là 1 biến được gán value từ return //SOÃI CA, RE-USE CAO NHẤT, NHƯNG VÀO LỆNH KHÁC</pre>		

© 2021 – Nguyễn Thế Hoàng – FPT University

3: Overload/Overloading

- Hiện tượng trong một class bất kì mà có hai hàm trùng tên nhau, nhưng khác phần tham số đầu vào, mà khác trên data type, không quan tâm tên biến, hiện tượng này được gọi là OVERLOAD/OVERLOADING – quá tải hàm.

```
public static void printIntegerList() { ... }
```

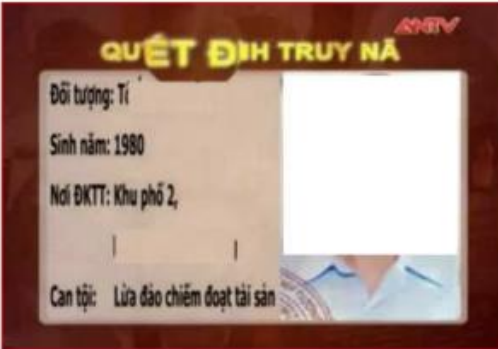
```
public static void printIntegerList(int n) {...}
```

Chapter 3: Object and Class

1: Object

03 Đối Tượng/Object





Đối tượng: những “thứ-object-thing” quanh ta, hữu hình/vô hình, chứa đựng nhiều thông tin giúp ta mô tả được chúng, nhận diện được chúng, phân biệt được chúng, đếm được chúng, (chạm) được chúng

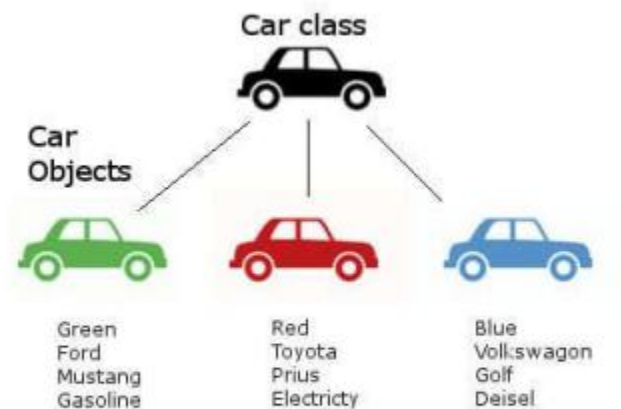
Đối tượng: chứa nhiều info được mô tả/nhận diện/phân biệt qua:

- Tên gọi tắt/định danh (biến phức tạp/biến object)
- Các đặc điểm (biến & value)
- Các hành vi/phương thức/hành động/behavior/method/hàm

© 2021 – Nguyễn Thế Hoàng – FPT University

2: Class

- Class giống như cái Khuôn, object là đối tượng/bức tượng được đúc ra từ Khuôn. Class là sự chia nhóm, gom nhóm các object tương đồng nhau về đặc điểm và hành vi.



Source: <https://breatheco.de/en/lesson/object-oriented-programming/>

- Trong Khuôn chứa các chi tiết mô tả object, ví dụ:
 - + Tượng có chiều cao, chiều cao đế.
 - + CMND/CCCD là Khuôn chứa thông tin mô tả công dân.
 - + CaSi là Khuôn chứa thông tin mô tả mọi object đi hát.
- Khuôn thì phải kín, code phải viết trong class.
- Khuôn là hình dáng của một object sẽ xuất hiện trong tương lai, chứa đựng đặc tính/đặc điểm và hành vi (xử lý thông tin - hàm).

```
public class Dog {
    private String name;
    private int yob;
    private double weight;
    ...
}
```

3: Constructor – Phễu

- Phễu là một hàm rất đặc biệt, nó xử lý: ép dòng dữ liệu/vật liệu nhận từ bên ngoài, chảy vào đúng chỗ bên trong vì Khuôn rỗng ở các chi tiết – có xử lý – tức là hàm theo quan điểm của lập trình.
- Mỗi lần sử dụng phễu, gọi phễu tương đương với việc đúc một bức tượng mới, tạo một object mới.
- Tên phễu phải trùng 100% tên class, chữ hoa từng đầu từ - quy ước.
- Hàm phễu không có giá trị trả về, không được void, vì void sẽ nhầm lẫn với ứng xử/hành động của object.

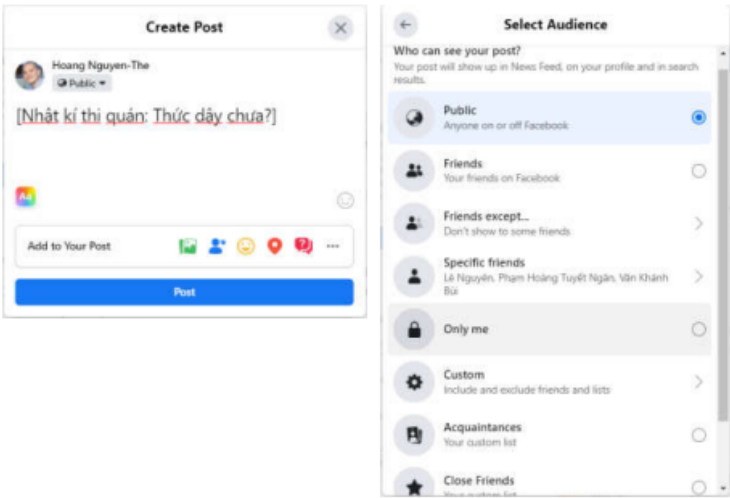
```
public Dog (String name, int yob, double weight) {
    this.name = name;
```



```
this.yob = yob;  
this.weight = weight;  
}
```

4: Access Specifier/Access Modifier

04 Access Specifier/Access Modifier/Quyền Được Sờ (tt)

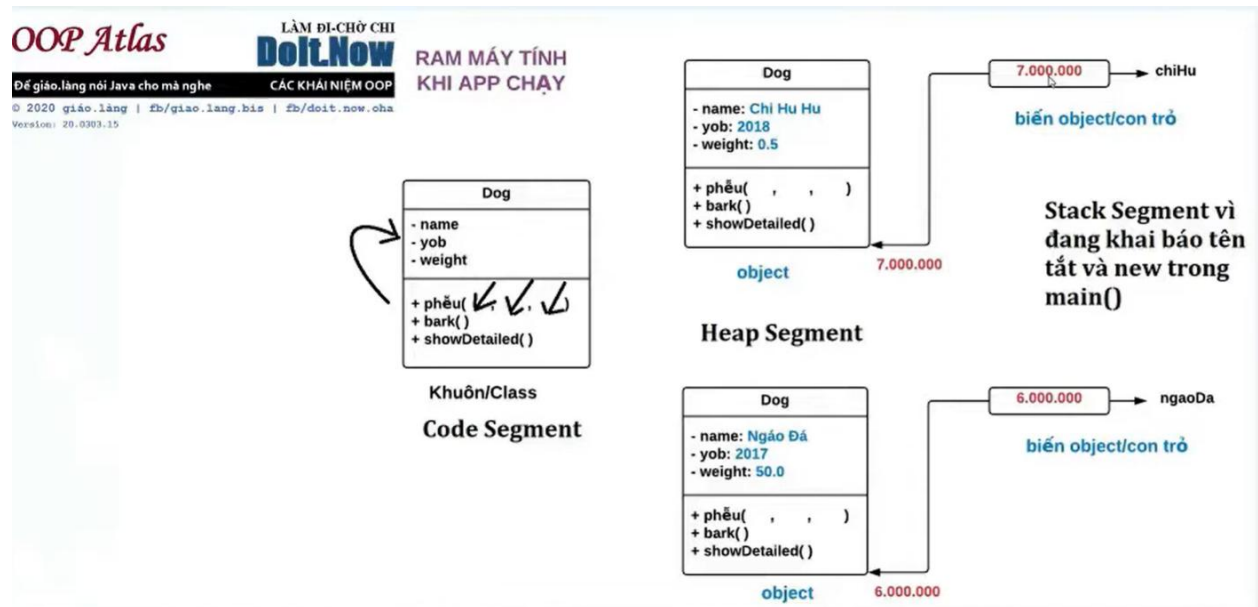


Quyền được “sờ” thông tin trong một khu vực/một object

- **private**: cấm sờ hiện vật.
Không thấy từ bên ngoài
- + **public**: cho không biểu không.
Cứ chạm nếu muốn
- # **protected**: họ hàng, bà con
giấu nhau làm gì
- ~ **default**: bạn thân cùng phòng

© 2021 – Nguyễn Thế Hoàng – FPT University

5: Object lưu trữ trong ram như thế nào?



- Ram có bốn vùng: Code Segment, Stack Segment, Heap Segment, và Data Segment.
- Object tồn tại hai vùng ram: một vùng ram là vùng new chính là object chứa các đặc điểm và hành vi, một vùng ram là biến con trỏ/biến object trỏ tới vùng new. Muốn chạm vào vùng new thì dùng biến object chấm. ON-OFF của biến con trỏ/biến object/value của biến con trỏ chính là tọa độ/địa chỉ vùng new.
- Biến nào khai báo trong hàm thì nằm trong Stack Segment.
- Object/vùng new nằm trong Heap Segment.

Chapter 4: Static

KHUÔN/CLASS

Student	
- id	
- name (static)	
- yob (non-static)	
- gpa	
+ Phieu (?, ?, ?)	
+ getX() / setX()	
+ toString()	
+ showProfile()	
+ sayHi() (static)	
+ ...	

```
public static void main(String[] args) {
    Student an = new Student("SE6789", "An Nguyễn", 2000, 9.0);
    Student binh = new Student("SE2204", "Binh Lê", 2001, 9.1);
}
```

OBJECT/INSTANCE

Student	
- id: SE6789	
- name	
- yob: 2000	
- gpa: 9.0	
+ Phieu (SE6789, ...)	
+ getX() / setX()	
+ toString()	
+ showProfile()	
+ sayHi()	
+ ...	

7.000.000 an

7.000.000

OBJECT/INSTANCE

Student	
- id: SE2204	
- name	
- yob: 2001	
- gpa: 9.1	
+ Phieu (SE2204, ...)	
+ getX() / setX()	
+ toString()	
+ showProfile()	
+ sayHi()	
+ ...	

6.000.000 binh

6.000.000

Student static zone	
- name: Binh Lê	
- =	
+ sayHi() {	
cout(???);	
}	
+ =	

GHI CHÚ VỀ STATIC:

- Vùng nhớ "xài/share" chung cho các object – WC tập thể
- Object nào phía bên kia cũng nghĩ là của "riêng mình" nhưng thực ra là của cả Họ Student
- Mọi object tăng để xài chung cho cả họ, nên biến/hàm ở vùng này thuộc "chiều trên", "đăng cấp" CLASS LEVEL, thuộc về mức Class
- Vì chung cho cả họ, nên "sở em" qua tên Class chấm trực tiếp, vì "em" này nào của riêng ai (object)
 - `Student.name` mới chuẩn com
 - `Student.sayHi()` mới chuẩn com
- Sở em qua tên-biến-object chấm thì không sai nhưng không nên cho "em" này đã là của chung rồi mở
 - `an.name an.sayHi()` không chuẩn com, chấm không bung lụa, phải tự gõ, không lỗi nhưng không nên theo kiểu này
- STATIC CHỈ CHƠI VỚI STATIC
- Dùng STATIC cho những gì không của riêng object mà là của cả họ
- Dùng STATIC cho những gì là tiện ích/Utility/Helper chỉ cần một vùng ram với mục đích cung cấp dịch vụ/service cho các object khác

GHI CHÚ VỀ NON-STATIC:

- Vùng nhớ "riêng" của từng object – WC riêng
- Thuộc "chiều dưới", ăn theo từng object cụ thể, OBJECT LEVEL, INSTANCE LEVEL, thuộc về mức object
- Vì của riêng từng object, nên phải xin object mới được "sở em", chấm qua tên object, "đền nhà ai nhà nấy rặng"
 - `an.showProfile()` chuẩn com
 - `binh.showProfile()` chuẩn com không ảnh hưởng nhà an chấm
- NON-STATIC CHƠI VỚI NON-STATIC VÌ CÙNG MỘT KHU VỰC RAM, CÙNG OBJECT
- NON-STATIC CHƠI LUÔN VỚI STATIC VÌ VẪN NGHĨ LÀ CỦA MÌNH (MÀ NAY LÀ CỦA CẢ HỌ) – mình có quyền xài WC công cộng mà
- Dùng NON-STATIC cho những gì thuộc từng riêng object, đặc điểm riêng của từng object
- BẠN KHÔNG MUỐN toàn bộ các sinh viên cùng tên, cùng năm sinh...đâu khi xài các đặc điểm id, name, yob... là static, hậu quả là thông tin ăn theo người cuối cùng được tạo ra, do chỉ có chung 1 vùng ram cho cái đám name, yob... đáng lẽ là của riêng từng ai đó, từng object sinh viên

Chapter 5: Input & Try-Catch

1: Input

- Hiện tượng bị trôi lệnh xảy ra khi lệnh nhập chuỗi đứng sau lệnh nhập số, lý do là vì khi user nhập xong, lệnh nhập số (`nextInt()`, `nextDouble()`) làm biếng không xóa rác trong buffer – bộ đệm bàn phím, tới lượt lệnh nhập chuỗi (`nextLine()`) chạy thì lấy rác còn trong buffer gán vào biến và không cho user nhập nữa.
- Có hai phương án giải quyết:
 - Tà chiêu, Âu Dương Phong solution:
System.out.print("Please input yob: ");

```
yob = sc.nextInt();  
sc.nextLine();  
+ Chánh đạo, Quách Tĩnh solution:  
yob = Integer.parseInt(sc.nextLine());
```

2: Try-Catch

```
while(true) {  
    try {  
        System.out.print("Please input yob: ");  
        yob = Integer.parseInt(sc.nextLine());  
        break;  
    } catch (Exception e) {  
        System.out.println("Do you know how to input an  
integer?");  
    }  
}
```

- Ta dùng try-catch khi:
 - + Convert data không thành công (từ chuỗi thành số).
 - + Chia cho 0 mà không chặn số bị chia qua if.
 - + Đi vượt quá số phần tử mảng (out of bound).
 - + Null pointer – chưa gán giá trị cho biến con trỏ/biến object mà đòi chấm.
 - + Kết nối mạng, truyền nhận dữ liệu qua mạng, database ở xa, server chết, mạng rớt.
 - + Sql injection.

- + I/O nhập xuất với tập tin, bắt buộc phải nhốt lệnh vào trong try-catch.
- + Và các trường hợp khác nữa.

Chapter 6: Array

1: Primitive Array

- Mảng là kỹ thuật khai báo nhiều biến hiệu quả (biến sẽ ứng với value nào đó) cùng kiểu, cùng lúc, ở sát nhau trong ram, và cùng tên.
- Khi các biến trùng tên, phải có cách để phân biệt biến này với biến kia, dùng tên phụ [index].
- Các cách khai báo mảng:
 - + Cách 1:
int arr[] = {0, 5, 10};
 - + Cách 2:
int[] arr = {0, 5, 10};
 - + Cách 3:
int arr[] = new int[10];
 - + Cách 4:
int[] arr = new int[] {0, 5, 10}
- Ngoặc [] là khai báo mảng, xin nhiều biến, còn () là gọi phễu của Khuôn heng. Cứ có new là có một vùng ram chà bá lửa, vùng ram này chứa 10 biến int, với giá trị mặc định của các biến là 0.
- Tên mảng là biến má mì, biến lớp trưởng, con trỏ, reference, trỏ vào đầu một dãy rất nhiều biến khác. Nó là biến lớp trưởng, biến

má mì vì nó quản lí nhiều biến khác. Hỏi biến lớp trưởng, biến má mì ơi, bạn đang quản lý bao nhiêu người, biến lớp trưởng, biến má mì chấm length.

```
for (int i = 0; i < arr.length; i++) {  
    System.out.print(arr[i] + “ “);  
}
```

```
for (int x : arr) {  
    System.out.print(x + “ “);  
}
```

2: Object Array

- Cách khai báo object array cũng tương tự như primitive array.

```
Student s[] = new Student[50];
```

+ Giá trị mặc định của các biến object Student là null, trở tới đây vùng ram, byte null, chứa toàn số 0, không có gì cả.

```
Student s2[] = new Student[] {  
    new Student(id, name, yob, gpa),  
    new Student(id, name, yob, gpa)  
};
```

- Nguyên tắc chơi với mảng object cấm tuyệt đối for hết nếu mảng chưa gán full vì phần tử chưa gán mặc định trở đầy ram, còn phần tử gán rồi thì chạy bình thường.
- Mảng object bắt buộc phải for đến count là số phần tử đã gán.

```
for (int i = 0; i < count; i++) {  
    s[i].showProfile();
```

}

Chapter 7: Inheritance

1: Giới thiệu tổng quát

- Các mức tư duy thiết kế:
 - + Object => Khuôn/Class là một mức nhận thức, thiết kế.
 - + Khuôn A, B, C thấy chúng có điểm tương đồng.
 - Đứng trên vai người không lồ: có sẵn rồi, làm thêm, phát huy thêm, ai đó dọn đường, ta đi tiếp.
 - Ngâm hiểu một câu chuyện: từ cái có sẵn, ta làm ra cái mới, phiên bản mới từ cái cũ. Cần có cái có sẵn, nền, để ta đi tiếp.
 - Muốn có object thì ta cần Khuôn X, Khuôn X làm từ đâu thì mất công, ta độ từ Khuôn Y nào đó. Khuôn X là bản điều chỉnh từ Khuôn Y, có nguồn gốc, dính dáng từ Khuôn Y, dùng lại những ưu thế, thứ có sẵn từ Khuôn Y.
- Kế thừa không phải là copy & paste, kế thừa là xài lại 100% không cần gõ lại code.
- Khi chơi với kế thừa, phải tôn trọng phễu gốc của Con. Ví dụ: Hình Vuông IS A (KIND OF) Hình Chữ Nhật.
 - + Hình Chữ Nhật gọi tên là class Cha, parent, super, base, phiên bản gốc.
 - + Hình Vuông gọi tên là class Con, child, sub, derived.

```
public class Rectangle {
    protected String owner;
    protected String color;
```

```
protected double width;  
private double length;  
  
public Rectangle(String owner, String color, double width,  
double weight) {  
    this.owner = owner;  
    this.color = color;  
    this.width = width;  
    this.length = length;  
}  
...  
}
```

```
public class Square extends Rectangle {  
    public Square (String owner, String color, double edge) {  
        super(owner, color, edge, edge);  
    }  
}
```

// super chính là new Rectangle(owner, color, width, length);
// super phải là câu lệnh đầu tiên trong hàm phễu của Con, có
Cha thì mới có Con.

2: Override

- Class Con có hàm trùng tên 100% với hàm Cha, Con qua mặt Cha, phủ quyết, vờ như không thấy hàm Cha.

- Đặc trưng cho 1 thứ gọi là tính đa hình, đa ánh xạ, transformer, POLYMORPHISM.

@Override

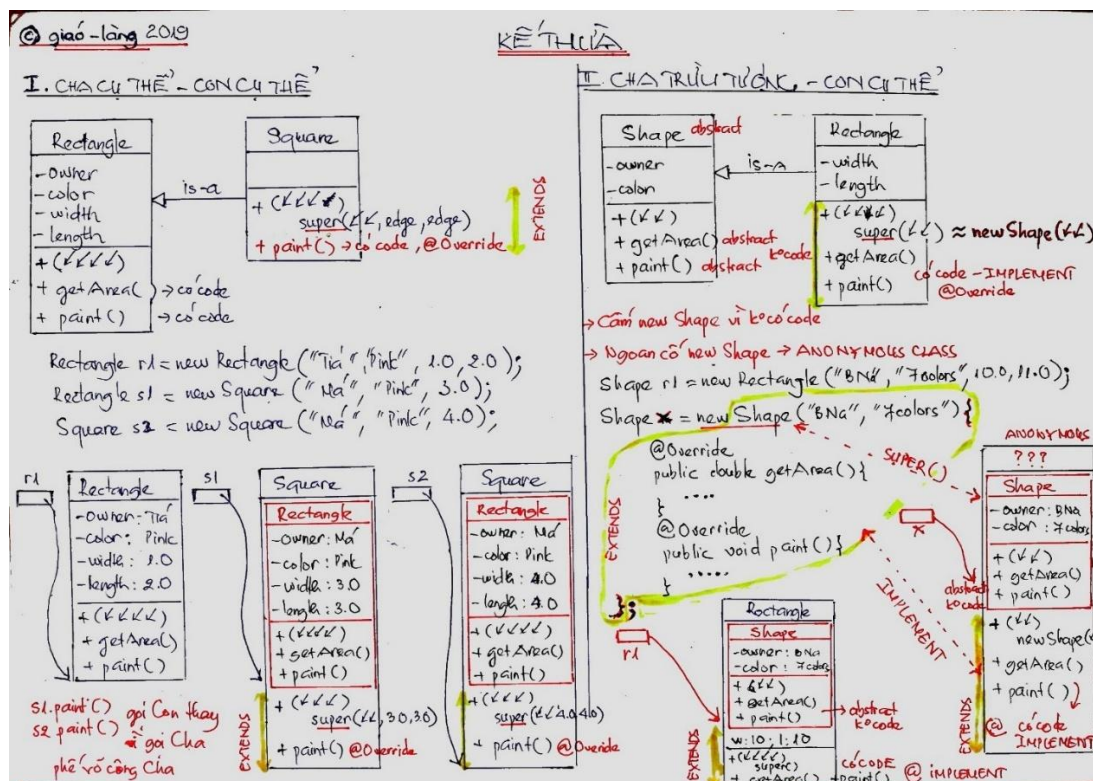
```
public void paint() {
```

```

        System.out.printf("|%-10s|%-15s|%-10s|%-4.1f|",
        "Square", owner, color, width);
    }
}

```

3: Cách tổ chức vùng ram khi chơi với Inheritance



- Vùng nhớ Con to/bao trùm vùng nhớ Cha với ý nghĩa: Con thừa hưởng tất cả đồ của Cha (trong vùng new có gì thì được xài cái đó). Cha là nền tảng/móng nhà/base để Con thoải mái vùng vẫy, mở rộng thêm (extends).
- Vùng nhờ Cha mở rộng/extends thêm một chút thành vùng nhớ Con. Cha sẽ là cánh chim đưa con bay thật xa. Ngôi nhà Cũ coi nới thêm gác lửng, extends thành nhà Mới.
- Vùng nhớ Con = vùng nhớ Cha + extends (mở rộng thêm).
 - + Vùng nhớ Cha: `super()` – thừa kế hết của Cha. [DI TRUYỀN].
 - + Extends (mở rộng thêm): `@Override` - Con trùng tên hàm Cha và Con có thêm điều khác Cha. [BIẾN DỊ].

- Chốt con trở:

- Khai Cha = `new Cha`, chấm xỏ ra gì? Xỏ ra tất cả của Cha.
- Khai Con = `new Con`, chấm xỏ ra gì? Xỏ ra tất cả của Con:
 - + Xỏ tất cả của Cha (thừa kế - di truyền).
 - + Xỏ tất cả phần `@Override`.
 - + Xỏ thêm của riêng Con (extends – biến dị). Con thêm hàm gì đó, không phải `@Override`, Cha không biết.
- Khai Cha = `new Con`, chấm xỏ ra gì?
 - + Xỏ tất cả của Cha (thừa kế - di truyền).
 - + Không xỏ `@Override` vì trùng tên hàm.
 - + Không xỏ ra những gì của riêng Con, chấp việc `new Con`, chấp việc trong ram đã có biến dị. Lý do: đang dùng con trở Cha, thì chỉ xỏ kiểu Cha, con trở chỉ trở đến vùng `super` (Cha nằm trong Con, không chịu nhìn xuống dưới extends).

- + Một thỏa hiệp khi chạy nếu thấy Con có hàm trùng tên thì ưu tiên Con.
- + Để thấy được hàm dị biệt của Con, phải drift con trở, kéo giãn sợi dây thun, ép kiểu, hai cách nhưng bản chất là một, kéo xuống đáy vùng new Con, chụp lại/giữ lại tọa độ new Con, rồi muốn chấm gì thì chấm, trở lại bình thường là Con.

Parent c = new Child(asset1, asset2);

// Cách 1

Child x = (Child)c;

x.showProfile();

// Cách 2

((Child)c).showProfile();

4: Abstract Class

- Ta cần chế tạo nhiều Khuôn mà lại chế tạo nhanh để từ đó ra object cho nhanh, vẫn là kế thừa mà thôi, vẫn phải dùng lại Khuôn nào đó. Vấn đề là dùng lại bao nhiêu Khuôn để làm ra nhiều Khuôn.
- Mục tiêu đa dạng object nhưng cần nhanh, mà lại chung phiên bản, tương đồng nhau.
- Ta cần một Khuôn Tổ, từ đó “đẻ” ra nhiều Khuôn con “khác nhau”.
- Đám khác nhau về chung một nhà, chung mảng, sắp xếp sẽ yên vui.

- Ví dụ: Vuông, Tròn, Chữ Nhật chung một nhà. Vuông, Tròn, Chữ Nhật IS A KIND OF ???
- Vuông, Tròn, Chữ Nhật, Tam Giác, ... is a kind of Shape (Hình Học), phạm là Hình Học sẽ có đường nét, màu sắc, diện tích, chu vi. Shape là Khuôn Tổ, từ đây ra cả đám Khuôn mới.
- Diện tích, chu vi của Hình Học chỉ là một khái niệm về một con số đại diện cho đường biên, bề mặt, tùy mỗi hình sẽ tính được khác nhau. Hình Học cũng là một khái niệm chung chung, chung chung thì sao mà tính được, cái chung chung vẫn ở đó, chúng ta gọi là trừu tượng – abstract.
- Cái gì tồn tại nhưng không chạm nắm được, chỉ mang ý nghĩa khái niệm, nói về một ý tưởng, cái đó hữu nhưng mà vô, nhìn thấy được nhưng không chạm nắm được – trừu tượng – abstract, ví dụ như ma quỷ, môn học khó nắm bắt, tình yêu,
- Diện tích, chu vi của Hình Học là trừu tượng, tồn tại để nói về thứ cần đo đạc nhưng chưa cụ thể, vì ngay cả Hình Học cũng là một khái niệm chưa cụ thể, biết là hình gì đâu. Vẽ cho tôi một cái hình, hỏi lại liền là hình gì? Chúng là những khái niệm sẽ được nạp vào đầu mình để biết rằng sẽ có gì đó diễn ra tiếp theo. Khái niệm – Abstract – Tính trừu tượng chính là nói về một ý tưởng.
- Class chứa hàm abstract, tức là hàm không có code, tức là ý tưởng cần có, thì bản thân class chưa hoàn hảo, chưa hành xử được, giống như bản concept, chỉ là ý tưởng mà thôi, do đó nếu class chứa hàm abstract thì class cũng phải là abstract.

```
public abstract class Shape {  
    protected String owner;  
    protected String color;
```

```

    public abstract double getArea();
    public abstract double getPerimeter();
}

```

- Class Con thừa kế từ class Cha, mà Cha này là abstract thì Con có hai lựa chọn:
 - + Con làm biếng, không thêm làm những điều Cha nói, không thêm viết code cho các hàm abstract của Cha. Lúc này, Con chứa các hàm abstract từ Cha. Mà class chứa hàm abstract thì chỉ là khái niệm, không hoàn hảo. Một class là abstract giống như bản concept, không thể vận hành một cách hoàn hảo được, do đó không thể new để tạo object được. Cha không new được do chưa hoàn hảo, Con cũng vậy, không new được, dòng họ vô sinh, không new để đúc object được.
 - + Con có trách nhiệm hoàn tất những điều Cha nói, viết code cho các hàm abstract của Cha, nếu không làm thì vô sinh, không new được do không hoàn hảo. Con nếu không muốn bị vô sinh thì phải viết code cho tất cả hàm abstract của Cha, implement all of the parent's abstract methods.

5: Anonymous Class

- Liên quan đến Hình Học, có xuất hiện những đối tượng lạ loài, không hình dạng quen thuộc, hình cắt ngẫu nhiên, không biết sắp xếp vào nhóm nào ngoại trừ nhóm Cha – Shape.
- Theo quan điểm object thì đã có object được tạo ra, vì có một hình mà, mà không biết Khuôn là gì, chỉ biết nó thuộc Khuôn Cha vì là Hình Học.

- Ta dùng kĩ thuật mượn gió bẻ măng, mượn Shape tạo object, vì ta thiếu một cái Khuôn Con cụ thể, nếu có Khuôn cụ thể (full code) new ngay Khuôn này ra hình mình cần.
- Tạo object mà không cần class riêng, mượn cha mà làm, class riêng mà không cần đề cập, gọi là class làm biếng, class không thêm làm cụ thể ra, class ẩn danh - anonymous.
- Vẫn new được object con của Shape mà không cần một class con cụ thể, vậy thì vùng ram con này chính là đại diện cho một class làm biếng làm riêng/đặt tên, class ẩn danh.
- Khai Cha new Cha, nếu Cha là abstract, mượn gió bẻ măng, Cha new đỡ cho Con, viết code on the go, đừng quên dấu chấm phẩy.

```

Shape vangLai = new Shape("VẮNG LAI", "GREEN") {
    @Override
    public double getArea() {
        return 50;
    }

    @Override
    public double getPerimeter() {
        return 40;
    }
};

```

Chapter 8: Interface

- Có hai hình thức gom nhóm:

- + Gom trên đặc điểm và hành vi – Inheritance (họ hàng bà con). Inheritance/Abstract Class: tên của một dòng họ nhiều con cháu chia sẻ đặc điểm và hành vi. “IS A KIND OF” – chặt chẽ trong phân loại (đặc điểm và hành vi).
- + Gom trên hành vi, tụ tập nhau làm gì đó – Interface – chung giao tiếp (hội nhóm). Interface: tên của một hội nhóm ăn chơi hành động. “IS A MEMBER OF” – lỏng lẻo việc kết hợp.
- + Vd: Bạn là Worker, Student, Lecturer, Employee, bạn cần cải thiện tiếng Anh, bạn tham gia vào CLB-Tiếng-Anh, bạn được đòi hỏi rằng chỉ cần speakEnglish() là đủ, không quan tâm bạn xuất thân từ đâu, chúng ta chơi chung, giao tiếp tiếng Anh. Worker, Student, Lecturer, Employee IS A MEMBER OF ENGLISH CLUB.
- Interface – CLB để mọi người tự chơi theo cách của mình, chơi – hành động là abstract. Chỉ nói nội quy CLB thôi heng, mọi người theo đó mà làm, tự mỗi người.
- Interface chứa hàm abstract, không cần ghi ra từ khóa abstract ở cả tên CLB và tên hàm, vì CLB hàm ý/ngầm hiểu là chỉ có nội quy chung chung để mọi người làm theo – implement, ngầm hiểu Interface là abstract, abstract là nêu ý tưởng.
- Interface cũng là một dạng của “kế thừa” hành động, chung hành động. Interface là bộ quy tắc hành xử/ứng xử/cam kết ai đó tham gia làm member thì phải theo hành động đã cam kết, nhưng sáng tạo trong hành động (implement).
- CLB không có đặc điểm, không có phễu/constructor – nhưng có hàm abstract. Nhưng khi new Con thì CLB vẫn sẽ được new để “Cha” sẽ là cánh chim, đưa Con extends @Override.

Member/Con vẫn có một vùng ram của CLB – chứa hàm abstract, con trở vẫn trở lưng chừng CLB. Vẫn theo mô hình ram của abstract class/Cha truyền thống. Con to hơn Cha/CLB về vùng ram. Super() Cha CLB là super() ngầm.

```
public interface DeadRacer() {  
    public double runToDead();  
    public void showHowToDead();  
}
```

```
public class Dog extends Pet implements DeadRacer {  
    ...  
    // implement các hàm abstract từ Cha Pet.  
    @Override  
    public double run() { ... }  
    @Override  
    public void showRecord { ... }  
  
    // implement các hàm abstract của CLB DeadRacer.  
    @Override  
    public double runToDead() { ... }  
    @Override  
    public void showHowToDead() { ... }  
}
```

- Trong Java, một class Con chỉ có tối đa một Cha – đơn kế thừa, single-inheritance. Nhưng Con có thể vào chơi nhiều CLB, chơi CLB nào thì có hành động của CLB đó.

- Con đóng N vai: vai dòng họ, vai CLB, giống ngoài đời. Nếu xui Cha, CLB có hàm trùng tên f(), thì Con có bị bối rối không? Con không bối rối vì f() của Cha và các CLB toàn là abstract, Con phải code @Override duy nhất hàm f(), không sợ ambiguous. Đường nào Con cũng @Override qua mặt Cha, CLB để xài Con, Cha và các CLB chỉ nói mà không làm.
- Anonymous:
 - + Object vắng lai tham gia CLB, vậy bạn cũng phải hành xử như việc bạn tham gia thường xuyên, phải có hành động tương ứng mới được tham gia, chỉ ngay lúc đó thôi, ON-THE-GO.
 - + Ghé ngang mà chơi, thì phải chơi/hành xử như người ta đang chơi.
 - + Bản chất ta có một nhóm nào đó trước đó, nhưng ta chỉ cần hòa nhập lúc này, chơi lúc này, tham gia lúc này, không cần bộc lộ rõ trước đó ta là ai, nhóm nào, chỉ cần biết join nhóm mới lúc này. Vd: CLBCờ x = new Student(...); // một Sinh Viên tham gia CLBCờ - code sẵn @Override.
 - // Gã ghé ngang, không chuyên nhưng ham vui, chơi thì chơi CLBCờ hangXom = new CLBCờ() {
 - @Override
 - // Mà ham chơi, ghé ngang tham gia, thì phải có hành động tương ứng;
 - }
- Ứng dụng của Interface:
 - + Giúp một class/object có nhiều cách hành xử khác nhau, tui là member của Hội nào thì tui có hành động của Hội đó, giúp cho một object/con người đóng nhiều vai, vai nào thì có hành động

tương ứng. Chính là một “hình dạng” của đa kế thừa – multi-inheritance.

+ Loose coupling (gắn kết lỏng lẻo) giúp cho cách tổ chức các class có ràng buộc nhau để làm nên câu chuyện lớn nhưng lại dễ dàng mở rộng và thay đổi.

+ Dùng Interface là cách viết code giúp linh hoạt, dễ dàng thay đổi code mà không phải sửa phần code cũ, dễ dàng mở rộng và thích ứng cho tương lai.

+ Interface là cách kết giao, mở rộng giao tiếp, kết nối với xung quanh.

+ Comparable/Comparator => dính đến loose coupling.

- Mục tiêu cuối cùng/tối thượng của Interface: chơi chung với nhau dù bạn khác biệt trong hành xử. Mỗi bạn đánh cờ khác nhau, nhưng cùng ván cờ. Mỗi card đồ họa khác nhau về năng lực xử lý, nhưng chung khe PCI-EX. Mỗi thiết bị điện khác nhau về công năng/tính năng sử dụng, nhưng chung ổ cắm 220V.

Chapter 9: OOP CONCEPTS / OOP PRINCIPLES / FUNDAMENTAL CONCEPTS OF OOP / FOUR PILLARS OF OOP

1: Abstraction – Trừu tượng hóa mọi vật

- Là cách tư duy, cách suy nghĩ, chuyển đổi, biểu diễn những thứ phức tạp trong cuộc sống thành những thứ đơn giản hơn nhưng vẫn diễn tả đúng ý nghĩa của sự vật ban đầu.
- Là cách nói về những thứ phức tạp trong cuộc sống thông qua một thứ khác đơn giản hơn, vẫn không làm mất đi ý nghĩa của thứ phức tạp ban đầu. Biểu diễn lại một thứ phức tạp ngoài đời để dễ hiểu hơn.
- Cách lấy ra những thông tin quan trọng nhất, tiêu biểu nhất của một object phức tạp ngoài đời để biểu diễn lại cho ai cũng hiểu được thứ phức tạp này.
- Using a simple thing to represent the complex thing. Extract the significant features/characteristics of a complex thing to form a simple/easy catching thing without losing the original meaning of the complex one.
- Vd: Sinh Viên ngoài đời rất phức tạp về info: tên, tuổi, địa chỉ, lịch trình di chuyển, gia cảnh, sở thích, chuyên ngành học, Trường đại học không cần nắm hết thông tin này, vẫn có thể quản lí được Sinh Viên. Tìm ra những info đặc trưng đủ để quản lí, biểu diễn/minh họa/mô tả (represent) một Sinh Viên. Sinh Viên

(id, email, tên, địa chỉ, phone, chuyên ngành), rút gọn/extract chỉ lấy những info quan trọng nhất, có ý nghĩa nhất để minh họa, biểu diễn một Sinh Viên bất kì, rút gọn từ N info còn vài info quan trọng – vừa xong thao tác trừu tượng hóa một bạn Sinh Viên, biến từ phức tạp thành đơn giản vẫn giữ nguyên ý nghĩa.

- Tìm đặc điểm quan trọng của một object, tìm đặc điểm cho một Class/Khuôn chính là Abstraction, biến đổi biểu diễn info không khiến mất gốc ban đầu.
- Mô tả lại một câu chuyện/thứ nào đó, để ai cũng hiểu được, tính trừu tượng/tính mô tả/ngắn gọn. Trừu tượng hóa tức là kể lại câu chuyện mô tả trong trí não kể lại cho ai đó nghe và người ta hiểu được.

2: Encapsulation – Cô lập đóng gói mọi thứ

- Nhận diện một đối tượng gồm đặc tính (biến - data) và hành vi (hàm – xử lý data), gom chung trong một đơn vị code/object/class.
- Gom xong giấu/che đi bằng thuật ngữ private, show ra nếu muốn theo cách public/protected.
- Vd:
 - + Giống y chang viên thuốc tây con nhộng – vỏ ngoài là protein – kiểu nhựa sinh học, tan khi uống vào, vụn tách viên thuốc bên trong thấy bột thuốc. Nhìn bên ngoài vỏ thuốc con nhộng, không thấy thuốc bên trong, ENCAPSULATE (v) gói lại che giấu, có thể trong suốt nếu muốn. Nhưng trong con nhộng chắc chắn chỉ có thuốc.

- + Tivi: che bằng cái vỏ nhựa, màn hình, ta chỉ tương tác qua remote, nút bấm, không thấy bên trong tivi ra sao, xử lý info + quét data bộ phim wifi/vô tuyến, không quan tâm đứng ngoài tivi giao tiếp ra sao, tivi chính là encapsulate.
- + ATM cũng vậy, không thấy khay tiền private, không thấy cục hút tiền, không thấy hành xử bên trong, chỉ public cái hành động bấm nút rút tiền, đóng gói cho việc rút tiền, không làm việc khác.
- Encapsulation chính là cách gom biến/data và hàm vào một đơn vị đối tượng/vật thể/object/class chỉ làm đúng việc của nó được chỉ định thiết kế, gom gì thì cần Abstraction để lấy cái đặc trưng. Vd: tivi xem phim, không gán thêm việc lò nướng.
- Abstraction là lấy ra đặc trưng, Encapsulation là gói trọn vẹn đúng thứ cần làm. Abstraction là nhận diện object, Encapsulation là tạo class instance variable + method().

3: Inheritance – Tái sử dụng / có xài luôn

- Reuse lại code không cần phải làm lại code, thậm chí không cần phải copy & paste.
- Con thừa kế di sản tài sản, di truyền sự thông minh và giỏi giang từ ba má!
- Tăng tốc trong quá trình viết code khi có đoạn code, object đã làm tốt việc của nó rồi, ta tận dụng để mở rộng thêm khả năng của object mới, dùng lại đồ object cũ + dị biệt ta làm thêm.
- Có thư viện viết code làm app cho nhanh, không làm lại từ đầu. Có thư viện, mở rộng thư viện cho nhu cầu riêng, đa năng sử dụng, tăng tốc viết code.

- Abstract class chẳng qua là cách gom, cách phân loại đối tượng mức cao, giống như phân loại sự vật, gom nhóm ngoài đời. Tận dụng, kế thừa thành quả của object có cái gì? Dùng lại nguyên tắc trừu tượng hóa – Abstraction + Encapsulation.

4: Polymorphism – Đa hình / đa ánh xạ - một ra nhiều

- Mùa đông 1946, Bác Hồ kêu gọi toàn quốc kháng chiến, ai có súng dùng súng, ai có dao dùng dao...
 - + Chỉ là một câu kêu gọi: Chiến(), nhân dân quật khởi đứng lên: dao, kiếm, liềm, súng, đa dạng cách giết giặc, đa dạng cách implements hành động Chiến().
 - + Từ một chữ Chiến() có N cách làm khác nhau, tính đa dạng, biến hình, đa sắc thái, đa nhân cách, đa hình, POLYMORPHISM.
- Đại ca kêu tôi nay giang hồ chợ lớn, đi chiến nha, đàn em đem hàng: mã tấu, dao, côn nhị khúc,
- Các vua Hùng đã có công dựng nước, Bác cháu Ta phải implements/@Override.
- Từ một hàm Cha, n Con @Override qua mặt, chính là tính đa hình, gọi Cha mà Con chạy.
- Đảm bảo tính đa dạng của các hành động vẫn chơi chung và thống nhất hành động qua tên gọi, @Override hiện thực cho tính đa hình.

Chapter 10: Collections Framework

1: Giới thiệu tổng quát

- Lưu trữ/xử lý với nhiều data/nhiều object thì:
 - + Ta cần mua cái túi, balo, tủ, phòng để đựng nhiều đồ. Mua cái túi, mua cái tủ chính là new cái Túi(), new cái Tủ(). Phải có một thùng chứa thật được tạo ra để ta bỏ đồ vô.
 - + Theo quan điểm lập trình: new để có vùng ram lưu trữ info + hàm xử lý info, theo Encapsulation. Object trong ram chính là tương đương món đồ thật ngoài đời, điện thoại trên tay bạn có màu sắc, giá tiền, hành động gọi nghe, chụp ảnh, bạn phải mua về, nhà sản xuất phải tạo ra, new ra bạn mới có để xài.
 - + Tủ là object chứa nhiều đồ đặc khác:
 - Mua Tủ: Tủ tuCuaTui = new Tủ();
 - Mua đồ add vô tủ: tuCuaTui.addMonDo(new/mua một món đồ thả vô tủ);

2: Các loại tủ

- Có hai loại tủ, dù tủ loại một hay hai thì đều chứa nhiều đồ: đồ để thật sự vào trong tủ hay chỉ giữ danh sách tham chiếu đều được. Tủ hồ sơ giữ danh sách tham chiếu, google meet giữ danh sách tham chiếu, tủ quần áo chứa đồ thật luôn – mảng primitive value lấy xài luôn.
- Có hai loại tủ:
 - Tủ loại 1: fix cứng số đồ để vô, chỉ để vô một số giới hạn món đồ. Chính là Mảng [số đồ để vô], mảng là khai báo nhiều biến

nhiều chỗ chứa [fix số chỗ]. Vd: mảng y chang xe ô tô chở khách 16 chỗ.

Student arr[] = new Student[100];

// sẽ có 100 chỗ ngồi cho SV à nhen, chưa có bạn nào cụ thể.

// bỏ đồ vào mảng, không cần dùng hàm, hành động của tử, cho sờ trực tiếp chỗ để.

arr[0] = món đồ bỏ vô, new món đồ bỏ vô;

- Tử loại 2: Co giãn số đồ để vô, thoải mái để đồ vô, cần thêm chỗ, có thêm chỗ không cần xin, cứ để đồ vô là thêm chỗ, giống nôi com thạch sanh, chỉ lưu tham chiếu, lưu con trỏ, add con trỏ vào, vùng new món đồ thoải mái nằm ở HEAP.
- + Không sờ [i] vào vị trí chỗ, xin qua dây kéo, khóa, mở ra nhét đồ vô.

tuCuaTui.add(món đồ bỏ vô) // bỏ đồ vô qua hành động của tử, không sờ trực tiếp ngăn chứa [i], bỏ vô tự tử lo bên trong.

+ Phân loại cao hơn cho tử/giỏ cụ thể - Cha – Abstract/Interface

List	Set	Map
ArrayList, Vector	HashSet, TreeSet	HashMap, TreeMap
- Danh sách. - Vào thứ tự nào ra thứ tự nấy, mảng nâng cao, co giãn kích thước, vào trước, ngồi [0], vào sau, ngồi [1]. - Ngoài đời, thống kê số vé bán, số lượt người mua, số lượt người ghé thăm, một người có thể đếm nhiều lần. - Khai báo:	- Tập hợp. - HashSet: đồ đưa vào lộn xộn thứ tự. - TreeSet: sắp xếp tự động.	- Bộ ánh xạ, từ này ra kia. - Key, món đồ - value

<pre>List arr = new ArrayList(); // đựng lộn xộn con trỏ. List<Student> arr = new ArrayList(); List<Student> arr = new ArrayList<>(); List<Student> arr = new ArrayList<Student>();</pre> - Thêm món đồ vô tủ: arr.add(new (...)); - Lấy món đồ ra: arr.get(i); - Lấy kích thước tủ, có thể for hết vì không sợ bị null pointer do add() đến đâu thêm con trỏ đến đó: arr.size(); - Xóa một phần tử trong tủ: arr.remove(i); - Thay đổi giá trị của phần tử: arr.set(i, new(...)); // Phần tử thứ i được gán một tọa độ mới.	- Không được lặp lại món đồ lần thứ hai. - Đếm số người thực sự xem phim, không đếm lượt, mỗi người xuất hiện một lần, không hơn dù bạn xem 2 3 n lần. - Khai báo: Set<Student> arr = new HashSet(); - Chỉ có add() không có get(), muốn lấy info trong Set chỉ có cách for each.	
---	---	--

Chapter 11: Wrapper Class

1: Giới thiệu tổng quát

- Ngữ cảnh:
Số 1, 2, 3, 5, 7, -10 – kiểu int
- Mô tả/đặc điểm:

- + Giá trị/đại lượng: 1, 2, ...
- + Nguyên con, trọn vẹn không lẻ miếng nào về giá trị.
- Mô tả hành động:
 - + $1 < 2$; $3 > 0$; so sánh các đại lượng của nhau.
 - + Giá trị tính theo vị trí (220 - số 2 đứng ở hàng trăm hay hàng chục sẽ khác nhau về giá trị).
- Hóa ra cái đám ở trên có chung đặc điểm là không lẻ miếng nào, mang giá trị còn so sánh được lẫn nhau => chúng thuộc nhóm Integer, nhóm những con số Nguyên hẳn không lẻ miếng nào.

```

public class Integer {
    private int value;

    public Integer (int value) {
        this.value = value;
    }

    // get() set()
    // Hàm siêu đặc biệt
    public static int parseInt(String num) {
        // biến chuỗi thành số để đem xài;
    }
}

```

Integer n = new Integer(5);

Integer n = 5; // new ngầm.

// Tồn 2 vùng ram, con số 5 primitive được nhét/gói vào 1 vùng ram bự hơn, vùng clone Khuôn Integer.

// Integer được gọi là class Gói – Wrapper Class, gói lại các con int primitive.

- Có tổng cộng 8 cái Wrapper Class/Class Trai Bao ứng với 8 primitive data type:

Byte – byte

Short – short

Integer – int

Long – long

Double – double

Float – float

Boolean – boolean

Character – char

2: Boxing & Unboxing

- Cơ chế boxing/đóng hộp: khi ta gói cái primitive thành object.

Integer n = 2003;

// CPU xử lý giống nhau, new ngầm Integer(), nhét vào trong lõi Integer vùng new á, con số int 2003 core/lõi.

- Unboxing: khi ta mở hộp, lấy value lõi để gán ra primitive, vẫn tương thích, phù hợp. Unboxing: từ object thành primitive.

int n2 = n;

3: Immutable Class & Mutable Class

Integer n = 2003;

n = 2000;

- Điều gì diễn ra trong ram? n trở vùng new mới Integer (lỗi 2000), không quan tâm vùng new cũ (lỗi 2003). Integer không có cơ chế set() lại cái gì đã đổ lúc new, không cho phép thay đổi value lỗi, tức là read-only, chỉ đọc, không cho chỉnh sửa.
- Một class bất kì nào khác, kể cả Integer, mà chỉ có hàm get(), không có hàm set(), không có hàm cho thay đổi bên trong, ngoại trừ lúc đúc, class read-only đặc điểm, thì class này được gọi là Immutable Class.
- Class nào mà cho phép thay đổi value đã được đúc, gọi là Mutable Class.

4: So sánh hai object

- So sánh hai object thì phải vào bên trong vùng new rồi chấm để lấy value để so sánh thì mới hợp lí.
- Có bằng nhau không? Có equals nhau? true or false.
- Có so với nhau như thế nào? Có compareTo nhau?
 - + Thua: $-1 < 0$.
 - + Bằng: 0 .
 - + Thắng: $1 > 0$.

Integer n1 = 2000;

Integer n2 = 2003;

n1.equals(n2); // false

n1.compareTo(n2); // -1

5: Pool

- Pool là một vùng ram chứa các vùng new của Wrapper Class, String, giống như hồ bơi ngoài trời, nhiều object tắm chung, chỉ áp dụng kỹ thuật hồ bơi trong một số tình huống đặc biệt, không nhầm với static.
- Ví dụ:
 - **Integer:**

```
Integer n1 = 127;
Integer n2 = 127;
Integer n3 = new Integer(127);
n1 == n2; // true
n1 == n3; // false
```

 - True: $[-128, 127]$, $[-2^7, 2^7 - 1]$ – khoảng giá trị của byte.
 - Chờ new nằm trong đoạn -128 đến 127 thì mới xài Pool.
 - Có 256 vùng new $[-128, 127]$ dùng chung cho n biến Integer, nếu xài kiểu boxing new nằm.
 - **String:**

```
String s1 = "HELLO";
String s2 = "HELLO";
String s3 = new String("HELLO");
String s4 = "Hello";
s1 == s2; // true
s1 == s3; // false
s1 == s4; // false
```

 - Chuỗi new nằm là pool.
 - Pool phân biệt hoa thường.